

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory

Experiment 3

**Implementation of a Convolutional Neural Network (CNN) for
Multi-Class Image Classification**

Degree & Branch	B.Tech Artificial Intelligence & Data Science
Semester	V
Subject Code & Name	CS3807 – Deep Learning Laboratory
AY	2026–27

1 Objective

Implement a Convolutional Neural Network (CNN) with TensorFlow/Keras on the CIFAR-10 Dataset. Learn the basic building blocks of CNNs, including convolutional layers, activation functions, pooling layers, flattening, and fully connected layers, and gain hands-on experience with feature-map visualization, model construction, training, and performance evaluation for multi-class image-classification tasks.

2 Background Theory

A CNN learns hierarchical spatial features of an image by repeated application of learnable convolution kernels and non-linear activation, followed by spatial down-sampling (pooling).

2.1 Convolution Operation

For an input feature map X and a kernel K of size $k \times k$, the output feature map at position (i, j) for a given filter is

$$Z(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X(i+m, j+n) K(m, n) + b \quad (1)$$

followed by a non-linear activation $A = f(Z)$, where f is typically the ReLU function $f(z) = \max(0, z)$.

2.2 Output Feature Map Size

For an input of size $W \times W$, kernel size k , padding p , and stride s , the output spatial dimension is

$$W_{out} = \left\lfloor \frac{W - k + 2p}{s} \right\rfloor + 1 \quad (2)$$

With `padding = 'same'` and stride 1, the convolution preserves the spatial dimensions of the input ($W_{out} = W$), so down-sampling in this network is performed entirely by the pooling layers.

2.3 Pooling

Max-pooling with a 2×2 window and stride 2 decreases the spatial resolution of the feature map by a factor of 2,

while retaining the largest activation in each window, thus reducing computation and achieving a limited form of translation invariance.

2.4 Classification Head

After the convolutional blocks, the final feature maps are flattened into a single vector and passed through fully connected (dense) layers. The output layer uses the Softmax function

$$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (3)$$

to produce a probability distribution over the 10 classes, and the network is trained by minimizing the categorical cross-entropy loss

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i) \quad (4)$$

3 Dataset

Dataset	CIFAR-10
Training Images	50,000
Testing Images	10,000
Classes	10 (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)
Image Size	$32 \times 32 \times 3$ (RGB)

```
Train images shape: (50000, 32, 32, 3)
Train labels shape : (50000, 1)
Test images shape : (10000, 32, 32, 3)
Test labels shape : (10000, 1)
```

4 Image Preprocessing

CIFAR-10 images were stored as 32×32 RGB arrays with pixel values in $[0, 255]$. Preprocessing involved loading the dataset, visualizing the images, normalizing the pixel values to $[0, 1]$ by dividing by 255, and flattening the label arrays for indexing. Since the labels were used directly in a sparse categorical cross-entropy loss, there was no need to one-hot encode them.

```
Before preprocessing:
X_train: (50000, 32, 32, 3) dtype: uint8

After preprocessing:
X_train_norm: (50000, 32, 32, 3) dtype: float32
```

5 Model Construction

The CNN was constructed with three convolutional blocks (each a 3×3 convolution with 'same' padding and ReLU activation, followed by 2×2 max-pooling), a flatten layer, a 128-unit fully connected layer with a dropout of 0.3 for regularization, and a 10-unit Softmax output layer.

```
def build_cnn():
    model = models.Sequential([
        layers.Conv2D(32, (3, 3), activation='relu', padding='same',
                      input_shape=(32, 32, 3), name='conv_1'),
        layers.MaxPooling2D((2, 2), name='pool_1'),

        layers.Conv2D(64, (3, 3), activation='relu', padding='same', name='conv_2'),
        layers.MaxPooling2D((2, 2), name='pool_2'),

        layers.Conv2D(128, (3, 3), activation='relu', padding='same', name='conv_3'),
        layers.MaxPooling2D((2, 2), name='pool_3'),

        layers.Flatten(name='flatten'),
        layers.Dense(128, activation='relu', name='fc_1'),
        layers.Dropout(0.3, name='dropout'),
        layers.Dense(10, activation='softmax', name='output')
    ])
    return model
```

5.1 Model Summary

Layer (type)	Output Shape	Param #
conv_1 (Conv2D)	(None, 32, 32, 32)	896
pool_1 (MaxPooling2D)	(None, 16, 16, 32)	0
conv_2 (Conv2D)	(None, 16, 16, 64)	18,496
pool_2 (MaxPooling2D)	(None, 8, 8, 64)	0
conv_3 (Conv2D)	(None, 8, 8, 128)	73,856
pool_3 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
fc_1 (Dense)	(None, 128)	262,272
dropout (Dropout)	(None, 128)	0
output (Dense)	(None, 10)	1,290

Table 1: Layer-wise output shapes and trainable parameters.

Total params: 356,810 (1.36 MB)

Trainable params: 356,810 (1.36 MB)

Non-trainable params: 0 (0.00 B)

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv_1 (Conv2D)	(None, 32, 32, 32)	896
pool_1 (MaxPooling2D)	(None, 16, 16, 32)	0
conv_2 (Conv2D)	(None, 16, 16, 64)	18,496
pool_2 (MaxPooling2D)	(None, 8, 8, 64)	0
conv_3 (Conv2D)	(None, 8, 8, 128)	73,856
pool_3 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
fc_1 (Dense)	(None, 128)	262,272
dropout (Dropout)	(None, 128)	0
output (Dense)	(None, 10)	1,290

```
Total params: 356,810 (1.36 MB)
Trainable params: 356,810 (1.36 MB)
Non-trainable params: 0 (0.00 B)
```

Figure 1: Console output of `model.summary()` for the baseline CNN.

6 Training Configuration

The model was compiled with the Adam optimizer, sparse categorical cross-entropy loss, and accuracy as the evaluation metric, and trained for 15 epochs with a batch size of 64, using the CIFAR-10 test split as validation data.

```
cnn_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

BATCH_SIZE = 64
EPOCHS = 15

history = cnn_model.fit(
    X_train_norm, y_train_flat,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(X_test_norm, y_test_flat),
    verbose=1
)
```

Optimizer	Adam
Loss Function	Sparse Categorical Cross-Entropy
Batch Size	64
Epochs	15
Dropout Rate	0.3

7 Source Code

The complete source code for this experiment is available at:

https://github.com/tonypanda7/DeepLearning_Lab

8 Mandatory Plots and Results

8.1 Sample Images

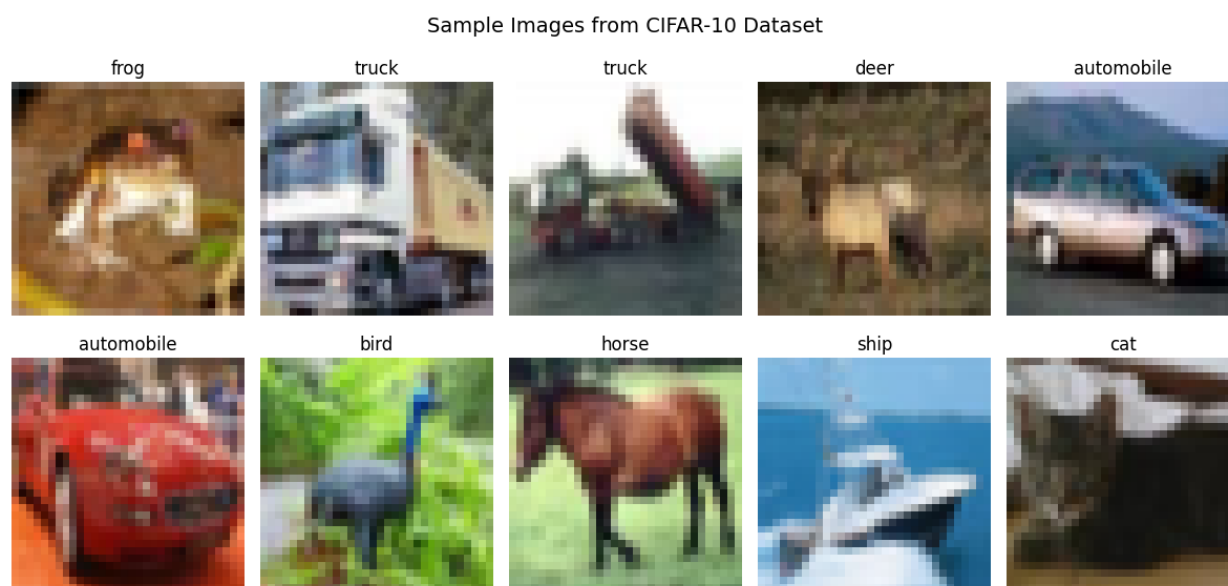


Figure 2: Sample images from the CIFAR-10 training set.

Interpretation: The sample grid shows ten randomly selected training images, which represent all object categories; this confirms that the database contains low-resolution (32×32) colour photos of common objects and animals that are varied enough to require careful analysis before classification.

8.2 Class Distribution

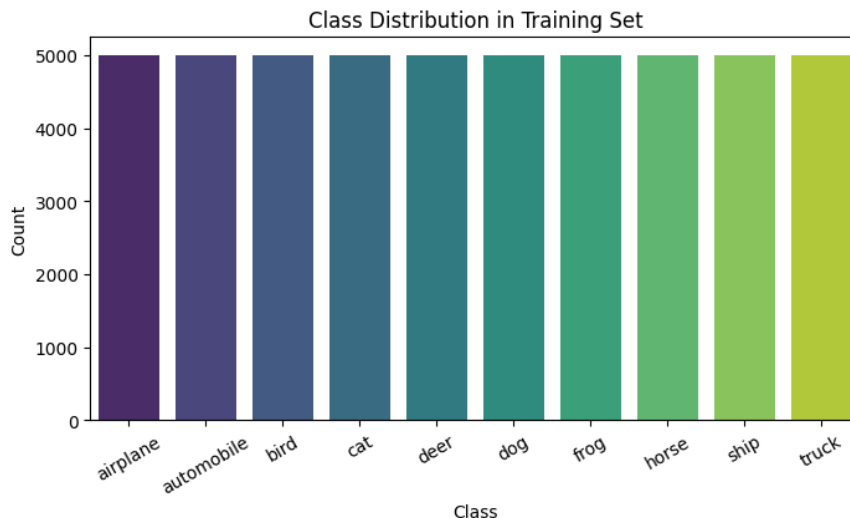


Figure 3: Class distribution in the CIFAR-10 training set.

Interpretation: All ten classes have 5,000 training images each, which makes the training set perfectly balanced. As a result, no class-weighting or resampling was necessary, and any differences in the performance for different classes are due to the visual ambiguity of the class or its similarity to other classes.

8.3 Training and Validation Curves

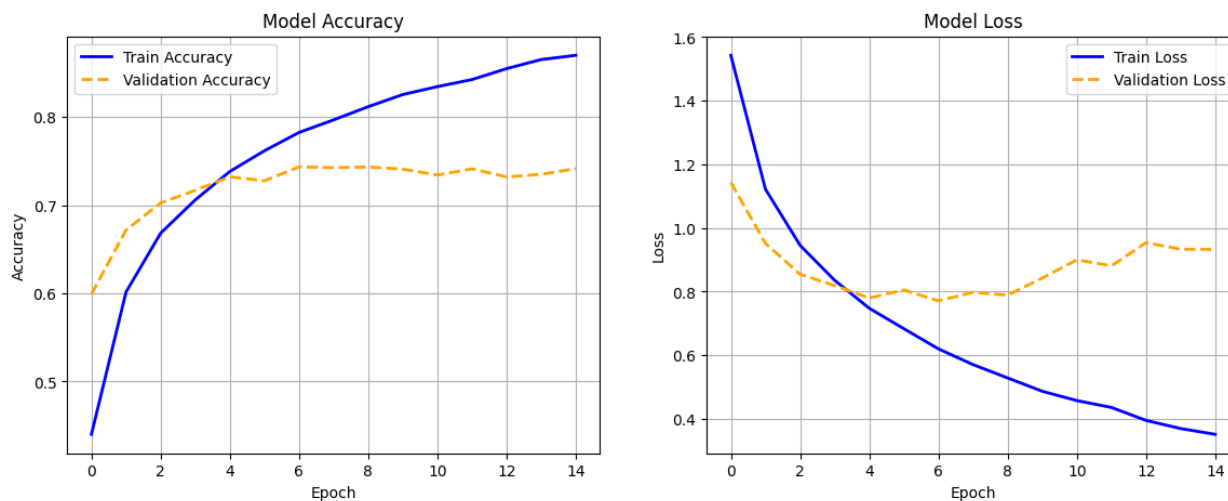


Figure 4: Training/validation accuracy (left) and loss (right) across 15 epochs.

Interpretation: As can be seen from the plot above, the training accuracy tends to increase steadily reaching about 87% while the validation accuracy plateaus around 74%. The training loss decreases steadily while the validation loss decreases up to epoch 6-7 and then starts increasing. Thus, the gap between the training and validation plots suggests that the model begins to overfit after epoch 6-7.

to overfit after epoch 6 which is evident from the validation loss plot peaking at epoch 6-7, which is only partially addressed by the dropout layer.

8.4 Feature Map Visualization

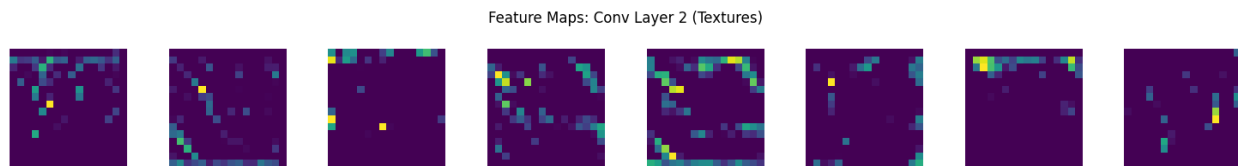


Figure 5: Feature maps extracted from `conv_2` for a sample test image, illustrating mid-level texture features.

Interpretation: The feature maps of the second convolutional layer reveal localized edge or texture responses, as opposed to the whole-object responses of the previous layer, which is in line with the CNN’s hierarchical feature-learning structure: the network’s first layers tend to learn features typical of low-level visual cues, such as colours and edges, while the later ones (in this case, the deepest convolutional layer, `conv_3`) are supposed to learn higher-level features, such as object-specific patterns.

8.5 Classification Report

```

... 313/313 ----- 2s 4ms/step
===== Classification Report =====
              precision    recall  f1-score   support

   airplane      0.70      0.86      0.77      1000
  automobile      0.88      0.85      0.86      1000
         bird      0.62      0.68      0.65      1000
          cat      0.60      0.49      0.54      1000
         deer      0.69      0.71      0.70      1000
          dog      0.60      0.68      0.64      1000
         frog      0.84      0.76      0.80      1000
        horse      0.81      0.76      0.79      1000
          ship      0.86      0.82      0.84      1000
         truck      0.86      0.80      0.83      1000

   accuracy                   0.74      10000
  macro avg      0.75      0.74      0.74      10000
 weighted avg      0.75      0.74      0.74      10000

```

Figure 6: Precision, recall, and F1-score per class on the CIFAR-10 test set.

Interpretation: The model achieves an overall test accuracy of 0.74. Vehicle and man-made object classes (automobile, ship, truck, airplane) attain the highest F1-scores (0.77–0.86), while animal classes – particularly cat (F1 = 0.54) and dog (F1 = 0.64) – are the hardest to classify, reflecting the greater intra-class variability and visual overlap between similar animal categories.

8.6 Confusion Matrix

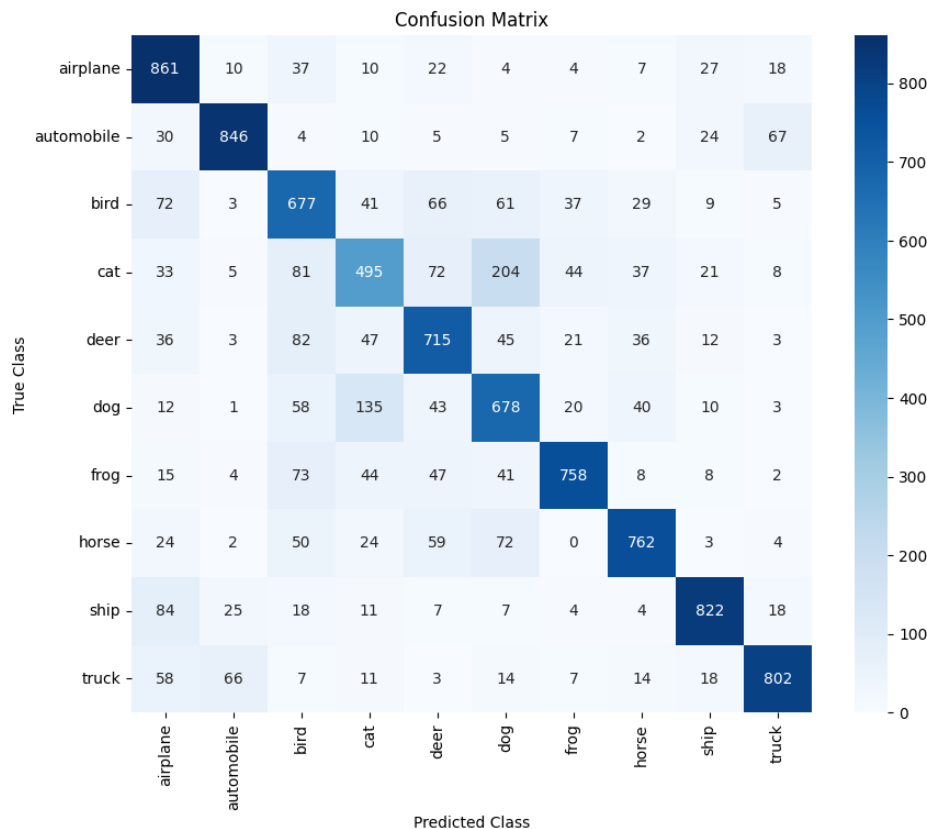


Figure 7: Confusion matrix on the CIFAR-10 test set.

Interpretation: The matrix on the confusion matrix section has a clear diagonal, which implies that the majority of the test images were correctly classified. Most confusions seem to be between cat and dog (204 cats were classified as dogs) and to a lesser extent, between bird, deer, and dog, probably due to the similarity of their natural background and body shape. Meanwhile, vehicle classes such as automobile, ship, and truck were rarely confused with animals, suggesting that the network can effectively differentiate between artificial and organic objects, despite its limited capacity to distinguish similar animals.

9 Discussion and Conclusion

Discussion. The baseline CNN, which comprises three convolutional blocks with 32, 64, and 128 filters, respectively, and a 128-neuron dense layer with 0.3 dropout, was trained for 15 epochs with Adam as an optimizer using a batch size of 64. The model achieved a decent accuracy of around 87% on the training set but only 74% on the test set, and the validation loss started to increase from epoch 6-7 while the training loss decreased – a telltale sign of overfitting. This is not surprising given that the fully connected classifier is relatively small (356,810 parameters) and was trained on 32×32 images without data augmentation, so it is not surprising that it started to memorize some training examples after epoch 6-7 when the convolutional part already extracted most of the features. Indeed, the confusion matrix and classification report also suggest that the model tends

to confuse animal classes (cat, dog, bird, deer) with each other while achieving very high precision and recall for vehicle classes, which implies that the convolutional filters learned to differentiate between animals and vehicles at a high level but not the finer details within the animal category.

Conclusion. A baseline three-block CNN (32-64-128 filters, 356,810 parameters) trained for 15 epochs on CIFAR-10 achieved around 74

10 References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Krizhevsky, A. *Learning Multiple Layers of Features from Tiny Images* (CIFAR-10 Dataset).
4. TensorFlow/Keras Documentation.